

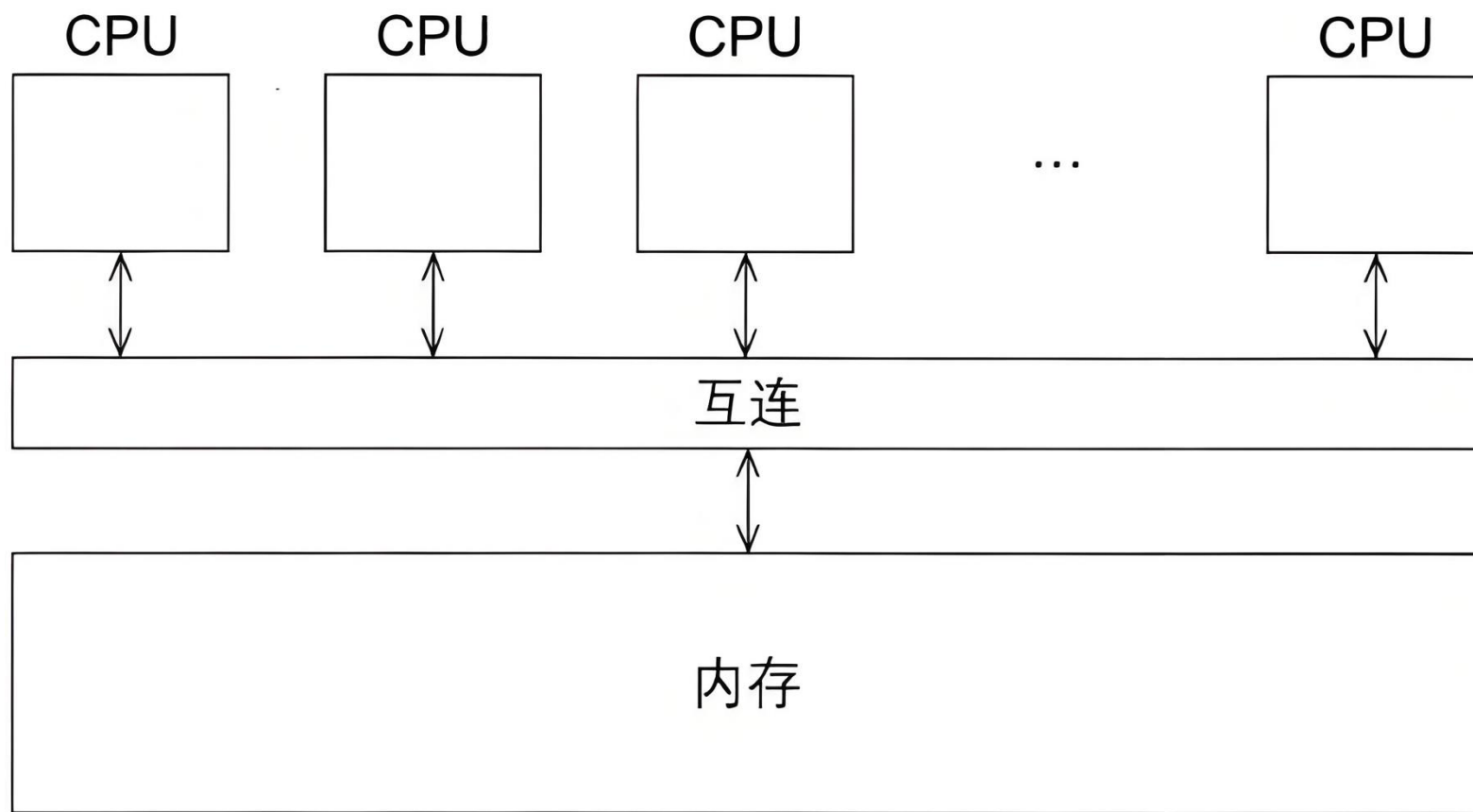
第4章 Pthreads共享内存编程

计算机与软件学院 陆克中

路线图

- 共享内存系统编程问题
- 临界区访问控制
- 线程同步
- 基于POSIX线程编程
- 互斥锁
- 生产者-消费者同步与信号量
- 栅栏与条件变量
- 读写锁
- 线程安全

共享内存系统



进程和线程

- 进程是正在运行（或挂起）的程序的一个实例。
- 线程类似于“轻量级”进程。
- 在共享内存程序中，单个进程可以拥有多个控制线程。

POSIX®线程

- 也称为Pthreads。
- 是类Unix操作系统的一项标准。
- 是一个可与C程序进行链接的**库**。
- 为多线程编程定义了应用程序编程接口（API）。

注意事项

- Pthreads API仅适用于**遵循POSIX标准**的系统，例如 Linux、macOS X、Solaris、HP-UX等。



Hello World!

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int thread_count; //全局变量: 所有线程都可以访问
void* Hello(void* rank) { //线程函数
    long my_rank = (long)rank;
    printf("Hello from thread %ld of %d\n",
        my_rank, thread_count);
    return NULL;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    thread_count = atoi(argv[1]); //从命令行中获
```

声明各类Pthreads函数、常量、数据类型等。

取线程的数量

```
thread_handles =
(pthread_t*)malloc(thread_count *
    sizeof(pthread_t));
for (thread = 0; thread < thread_count;
    thread++)
    pthread_create(&thread_handles[thread],
        NULL, Hello, (void*)thread);
printf("Hello from the main thread\n");
for (thread = 0; thread < thread_count;
    thread++)
    pthread_join(thread_handles[thread], NULL);
free(thread_handles);
return 0;
}
```

编译和运行

■ Windows

▶ 安装pthread-win32库

- <https://sourceware.org/pthreads-win32/>

▶ Visual Studio项目->属性

- C/C++->附加包含目录: C:\pthread\Pre-built.2\include
- 链接器->附加库目录: C:\pthread\Pre-built.2\lib\x64
- 链接器->输入->附加依赖项: pthreadVC2.lib
- pthreadVC2.dll文件放到程序运行时能找到的位置。

编译和运行

■ Linux

- ▶ C: `gcc -O3 -o pth_hello pth_hello.c -lpthread`
- ▶ C++: `g++ [-O3] [-g] [-Wall] -o pth_hello pth_hello.cpp -lpthread`

编译器优化

生成调试信息

开启编译器警告

链接Pthreads库

`./pth_hello <number of threads>`

`./pth_hello 1`

Hello from the main thread

Hello from thread 0 of 1

`./pth_hello 4`

Hello from the main thread

Hello from thread 2 of 4

Hello from thread 1 of 4

Hello from thread 0 of 4

Hello from thread 3 of 4

全局变量

- 可能会引入难以察觉、令人费解的错误！
- 仅在确实必要的场景下才使用全局变量。
 - ▶ 共享变量。



启动线程

- MPI中的进程通常由脚本启动。
- 而在Pthreads中，线程由可执行程序本身启动。

pthread.h

pthread_t

每个线程
一个对象

```
int pthread_create(  
    pthread_t*          thread_p          /* out */,  
    const pthread_attr_t* attr_p          /* in */,  
    void*               (*start_routine)(void*) /* in */,  
    void*               args_p            /* in */);
```

pthread_t对象

- 不透明类型
- 它们存储的实际数据与具体系统相关。
- 用户代码无法直接访问其数据成员。
- 不过，Pthreads 标准保证，pthread_t对象会存储足够的信息，以唯一标识它所关联的线程。

深入观察

- 第1个参数：必须在调用之前被分配。
- 第2个参数：不会使用，直接传递NULL。
- 第3个参数：线程要运行的函数。
- 第4个参数：指向应传递给start_routine函数的参数的指针。

```
int pthread_create(  
    pthread_t*          thread_p          /* out */,  
    const pthread_attr_t* attr_p          /* in */,  
    void*               (*start_routine)( void*) /* in */,  
    void*               args_p            /* in */);
```

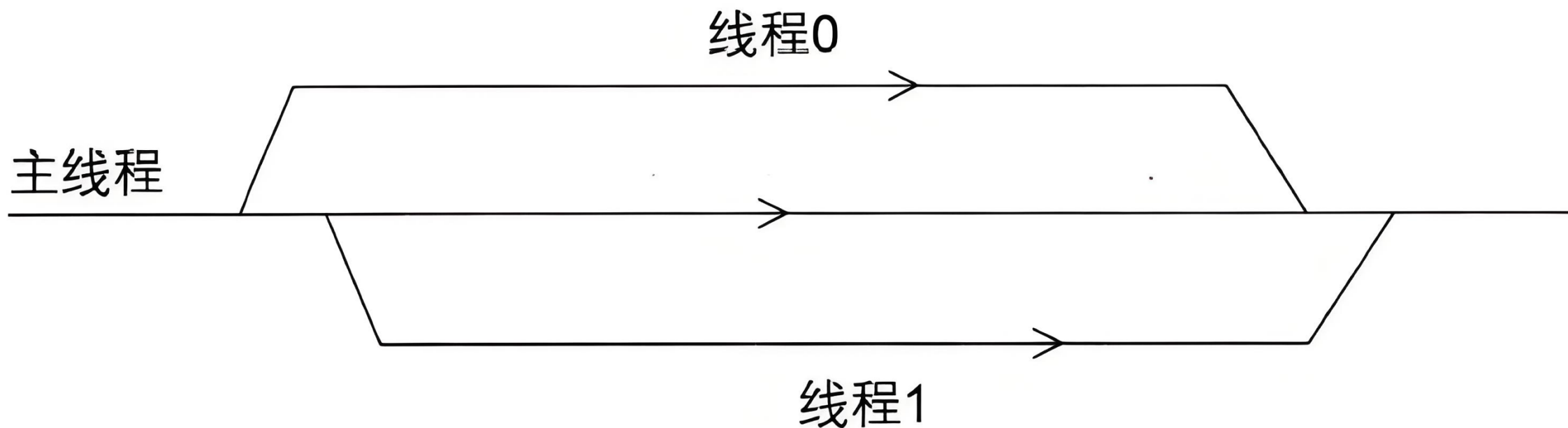
由pthread_create启动的函数

- 原型:

`void* thread_function(void* args_p);`

- 在C语言中, void*可以被强制转换为任意指针类型。
- 因此args_p可以指向一个列表, 其中包含线程函数thread_function所需的一个或多个值。
- 同样地, thread_function的返回值也可以指向包含一个或多个值的列表。

运行线程

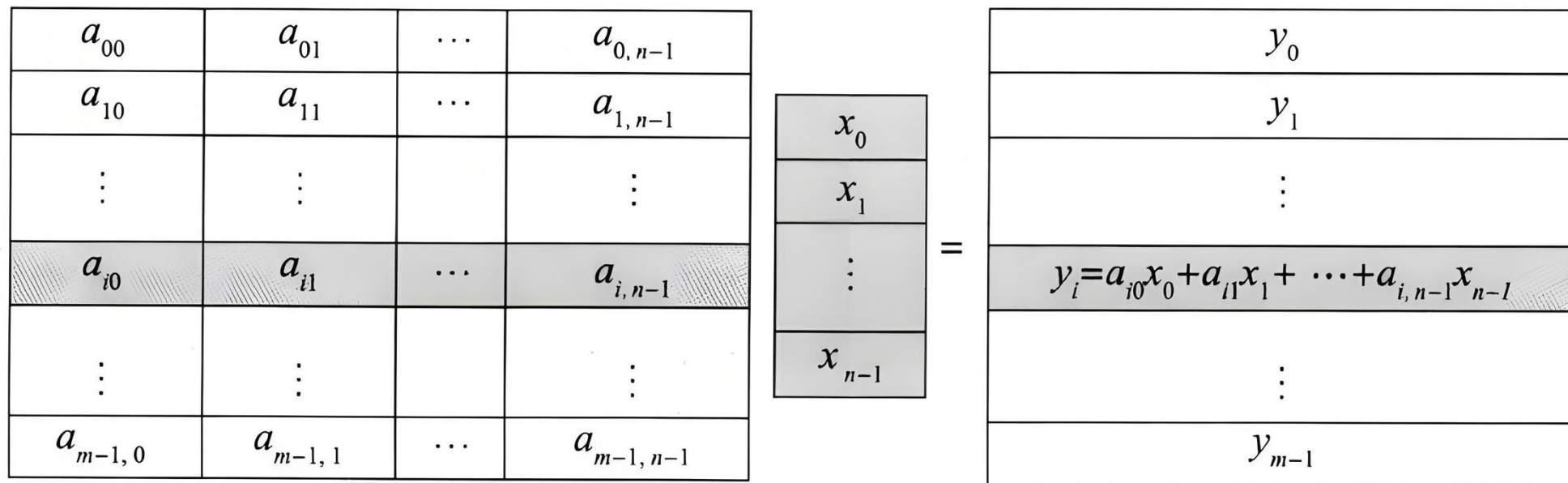


主线程创建并等待两个线程执行完毕。

停止线程

- 我们为每个线程调用一次pthread_join函数。
- 单次调用pthread_join会等待与该pthread_t对象关联的线程执行完毕。

基于POSIX线程的矩阵向量乘法



```

/* 对于A的每一行 */
for (i = 0; i < m; i++) {
    y[i] = 0.0;
    /* 对于行的每个元素和x的每个元素 */
    for (j = 0; j < n; j++)
        y[i] += A[i][j] * x[j];
}
    
```

$$y_i = \sum_{j=0}^{n-1} a_{ij} x_j$$

使用3个Pthreads线程

线程	y 的分量
0	y[0], y[1]
1	y[2], y[3]
2	y[4], y[5]

线程0

```
y[0] = 0.0;  
for (j = 0; j < n; j++)  
    y[0] += A[0][j]* x[j];
```

一般情况

```
y[i] = 0.0;  
for (j = 0; j < n; j++)  
    y[i] += A[i][j]*x[j];
```

Pthreads矩阵-向量乘法

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int thread_count;
const int m = 2520 * 4;
const int n = m;
double A[m][n];
double x[n];
double y[m];
void* Pth_mat_vect(void* rank) {
    long my_rank = (long)rank;
    int local_m = m / thread_count;
    int my_first_row = my_rank * local_m;
    int my_last_row = (my_rank + 1) * local_m - 1;
    for (int i = my_first_row; i <= my_last_row; i++) {
        y[i] = 0.0;
        for (int j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    }
    return NULL;
```

```

}
int main(int argc, char* argv[]) {
    long thread;
    double start, finish, elapsed;
    thread_count = atoi(argv[1]);
    pthread_t* thread_handles =
        (pthread_t*)malloc(thread_count * sizeof(pthread_t));
    start = clock();
    for (thread = 0; thread < thread_count; thread++)
        pthread_create(&thread_handles[thread], NULL,
            Pth_mat_vect, (void*)thread);
    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);
    finish = clock();
    elapsed = (finish - start) / CLOCKS_PER_SEC;
    printf("Elapsed time = %f seconds\n", elapsed);
    free(thread_handles);
    return 0;
}
```

临界区



估算 π

$$\pi = 4 \left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots + (-1)^n \frac{1}{2n+1} + \cdots \right)$$

```
double factor = 1.0;
double sum = 0.0;
for (i = 0; i < n; i++, factor = -factor) {
    sum += factor/(2*i+1);
}
pi = 4.0*sum;
```

用于计算 π 的线程函数

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <pthread.h>
long thread_count = 1;
long long n = 1000;
double sum = 0.0;
void* Thread_sum(void* rank) {
    long my_rank = (long)rank;
    double factor;
    long long i;
    long long my_n = n / thread_count;
    long long my_first_i = my_n *
my_rank;
    long long my_last_i = my_first_i +
my_n;
    if (my_first_i % 2 == 0)
        factor = 1.0;
    else
        factor = -1.0;
```

```
    for (i = my_first_i; i < my_last_i;
i++, factor = -factor) {
        sum += factor / (2 * i + 1);
    }
    return NULL;
}
int main(int argc, char* argv[]) {
    long thread;
    double start, finish, elapsed;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    if (argc > 2)
        n = atoi(argv[2]);
    pthread_t* thread_handles =
(pthread_t*)malloc(thread_count *
sizeof(pthread_t));
    start = clock();
    for (thread = 0; thread <
thread_count; thread++)
        pthread_create(&thread_handles[thr
```

```
ead], NULL, Thread_sum,
(void*)thread);
    for (thread = 0; thread <
thread_count; thread++)
        pthread_join(thread_handles[thread],
NULL);
    finish = clock();
    elapsed = (finish - start) /
CLOCKS_PER_SEC;
    sum = 4.0 * sum;
    printf("Estimate of pi    = %.15f\n",
sum);
    printf("Elapsed time = %f
seconds\n", elapsed);
    printf("Math library of pi =
%.15f\n", 4.0 * atan(1.0));
    free(thread_handles);
    return 0;
}
```

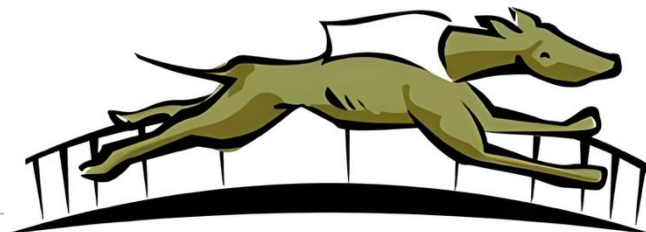
使用双核处理器

	n			
	10^5	10^6	10^7	10^8
π	3.14159	3.141593	3.1415927	3.14159265
一个线程	3.14158	3.141592	3.1415926	3.14159264
两个线程	3.14158	3.141480	3.1413692	3.14164686

请注意，随着 n 的增大，使用单线程得到的估算结果会越来越精确。

可能的竞争条件

时间	线程 0	线程 1
1	从主线程开始	
2	调用 Compute() 函数	从主线程开始
3	赋值 $y=1$	调用 Compute() 函数
4	将 $x=0$ 和 $y=1$ 放进寄存器	赋值 $y=2$
5	0 和 1 相加	将 $x=0$ 和 $y=2$ 放进寄存器
6	在内存位置 x 存储 1	0 和 2 相加
7		在内存位置 x 存储 2



忙等待

- 一个线程会反复检测某个条件，但在该条件达到预期值之前，实际上不会执行任何有效工作。
- 不过，要警惕优化编译器！

```
y = Compute(my_rank);  
while (flag != my_rank);  
x = x + y;  
flag++;
```

主线程将flag初始化为0

基于忙等待的Pthreads全局求和

```
void* Thread_sum(void* rank) {
    long my_rank = (long) rank;
    double factor;
    long long i;
    long long my_n = n/thread_count;
    long long my_first_i = my_n*my_rank;
    long long my_last_i = my_first_i + my_n;

    if (my_first_i % 2 == 0)
        factor = 1.0;
    else
        factor = -1.0;

    for (i = my_first_i; i < my_last_i; i++, factor = -factor) {
        while (flag != my_rank);
        sum += factor/(2*i+1);
        flag = (flag+1) % thread_count;
    }

    return NULL;
} /* Thread_sum */
```

基于忙等待的Pthreads全局求和

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <pthread.h>
long thread_count = 1;
long long n = 1000;
double sum = 0.0;
int flag = 0;
void* Thread_sum(void* rank) {
    long my_rank = (long)rank;
    long long my_n = n / thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i = my_first_i + my_n;
    double factor = my_first_i % 2 == 0 ? 1.0 : -1.0;
    for (long long i = my_first_i; i < my_last_i; i++, factor = -factor) {
        while (flag != my_rank);
        sum += factor / (2 * i + 1);
        flag = (flag + 1) % thread_count;
    }
    return NULL;
}
double Serial_pi(void) {
```

```
    double sum = 0.0;
    double factor = 1.0;
    for (long long i = 0; i < n; i++, factor = -factor)
    {
        sum += factor / (2 * i + 1);
    }
    return 4.0 * sum;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    double start, finish, elapsed;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    if (argc > 2)
        n = atoi(argv[2]);
    thread_handles =
        (pthread_t*)malloc(thread_count *
        sizeof(pthread_t));
    start = clock();
    for (thread = 0; thread < thread_count; thread++)
        pthread_create(&thread_handles[thread],
        NULL, Thread_sum, (void*)thread);
```

```
    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);
    finish = clock();
    elapsed = (finish - start) /
        CLOCKS_PER_SEC;
    sum = 4.0 * sum;
    printf("Multi-threaded estimate of pi = %.15f\n", sum);
    printf("Elapsed time = %f seconds\n", elapsed);
    start = clock();
    sum = Serial_pi();
    finish = clock();
    elapsed = (finish - start) /
        CLOCKS_PER_SEC;
    printf("Single-threaded estimate of pi = %.15f\n", sum);
    printf("Elapsed time = %f seconds\n", elapsed);
    printf("Math library estimate of pi = %.15f\n", 4.0 * atan(1.0));
    free(thread_handles);
    return 0;
}
```

循环后带临界区的全局求和函数

```
void* Thread_sum(void* rank) {
    long my_rank = (long) rank;
    double factor, my_sum = 0.0;
    long long i;
    long long my_n = n/thread_count;
    long long my_first_i = my_n*my_rank;
    long long my_last_i = my_first_i + my_n;

    if (my_first_i % 2 == 0)
        factor = 1.0;
    else
        factor = -1.0;

    for (i = my_first_i; i < my_last_i; i++, factor = -factor)
        my_sum += factor/(2*i+1);

    while (flag != my_rank);
    sum += my_sum;
    flag = (flag+1) % thread_count;

    return NULL;
} /* Thread_sum */
```

循环后带临界区的全局求和函数

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <pthread.h>
long thread_count = 1;
long long n = 1000;
double sum = 0.0;
int flag = 0;
void* Thread_sum(void* rank) {
    long my_rank = (long)rank;
    long long my_n = n / thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i = my_first_i + my_n;
    double factor = my_first_i % 2 == 0 ? 1.0 : -1.0;
    double my_sum = 0.0;
    for (long long i = my_first_i; i < my_last_i; i++,
        factor = -factor)
        my_sum += factor / (2 * i + 1);
    while (flag != my_rank);
    sum += my_sum;
    flag = (flag + 1) % thread_count;
    return NULL;
}
```

```
double Serial_pi(void) {
    double sum = 0.0;
    double factor = 1.0;
    for (long long i = 0; i < n; i++, factor = -factor)
    {
        sum += factor / (2 * i + 1);
    }
    return 4.0 * sum;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    double start, finish, elapsed;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    if (argc > 2)
        n = atoi(argv[2]);
    thread_handles =
        (pthread_t*)malloc(thread_count *
            sizeof(pthread_t));
    start = clock();
    for (thread = 0; thread < thread_count; thread++)
        pthread_create(&thread_handles[thread], NULL,
```

```
Thread_sum, (void*)thread);
    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);
    finish = clock();
    elapsed = (finish - start) / CLOCKS_PER_SEC;
    sum = 4.0 * sum;
    printf("Multi-threaded estimate of pi =
        %.15f\n", sum);
    printf("Elapsed time = %f seconds\n", elapsed);
    start = clock();
    sum = Serial_pi();
    finish = clock();
    elapsed = (finish - start) / CLOCKS_PER_SEC;
    printf("Single-threaded estimate of pi =
        %.15f\n", sum);
    printf("Elapsed time = %f seconds\n", elapsed);
    printf("Math library estimate of pi = %.15f\n",
        4.0 * atan(1.0));
    free(thread_handles);
    return 0;
}
```

互斥锁

- 处于忙等待的线程可能会持续占用CPU却不执行任何有效操作。
- 互斥锁（Mutex, mutual exclusion）是一种特殊类型的变量，可用于将临界区的访问权限限制为同一时间仅允许单个线程进入。

互斥锁

- 用于确保一个线程在执行临界区时“排斥”所有其他线程。
- Pthreads标准中包含一种专门用于互斥锁的数据类型：
`pthread_mutex_t`。



```
int pthread_mutex_init(  
    pthread_mutex_t*      mutex_p    /* out */,  
    const pthread_mutexattr_t* attr_p /* in */);
```

互斥锁

- 当Pthreads程序使用完互斥锁后，应当调用

```
int pthread_mutex_destroy(  
    pthread_mutex_t*  mutex_p    /* in/out */);
```

- 为了获得对临界区的访问权限，线程需要调用

```
int pthread_mutex_lock(  
    pthread_mutex_t*  mutex_p    /* in/out */);
```

- 当线程执行完临界区中的代码后，它应当调用

```
int pthread_mutex_unlock(  
    pthread_mutex_t*  mutex_p    /* in/out */);
```


使用互斥锁的全局求和函数

```
void* Thread_sum(void* rank) {
    long my_rank = (long) rank;
    double factor;
    long long i;
    long long my_n = n/thread_count;
    long long my_first_i = my_n*my_rank;
    long long my_last_i = my_first_i + my_n;
    double my_sum = 0.0;

    if (my_first_i % 2 == 0)
        factor = 1.0;
    else
        factor = -1.0;

    for (i = my_first_i; i < my_last_i; i++, factor = -factor) {
        my_sum += factor/(2*i+1);
    }
    pthread_mutex_lock(&mutex);
    sum += my_sum;
    pthread_mutex_unlock(&mutex);

    return NULL;
} /* Thread_sum */
```

使用互斥锁的全局求和函数

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <pthread.h>
long thread_count = 1;
long long n = 1000;
double sum = 0.0;
pthread_mutex_t mutex;
void* Thread_sum(void* rank) {
    long my_rank = (long)rank;
    long long my_n = n / thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i = my_first_i + my_n;
    double factor = my_first_i % 2 == 0 ? 1.0 : -1.0;
    double my_sum = 0.0;
    for (long long i = my_first_i; i < my_last_i; i++,
        factor = -factor)
        my_sum += factor / (2 * i + 1);
    pthread_mutex_lock(&mutex);
    sum += my_sum;
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

```
double Serial_pi(void) {
    double sum = 0.0;
    double factor = 1.0;
    for (long long i = 0; i < n; i++, factor = -factor) {
        sum += factor / (2 * i + 1);
    }
    return 4.0 * sum;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    double start, finish, elapsed;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    if (argc > 2)
        n = atoi(argv[2]);
    thread_handles =
        (pthread_t*)malloc(thread_count *
            sizeof(pthread_t));
    pthread_mutex_init(&mutex, NULL);
    start = clock();
    for (thread = 0; thread < thread_count; thread++)
        pthread_create(&thread_handles[thread], NULL,
```

```
Thread_sum, (void*)thread);
    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);
    finish = clock();
    elapsed = (finish - start) / CLOCKS_PER_SEC;
    sum = 4.0 * sum;
    printf("Multi-threaded estimate of pi =
        %.15f\n", sum);
    printf("Elapsed time = %f seconds\n", elapsed);
    start = clock();
    sum = Serial_pi();
    finish = clock();
    elapsed = (finish - start) / CLOCKS_PER_SEC;
    printf("Single-threaded estimate of pi =
        %.15f\n", sum);
    printf("Elapsed time = %f seconds\n", elapsed);
    printf("Math library estimate of pi = %.15f\n",
        4.0 * atan(1.0));
    pthread_mutex_destroy(&mutex);
    free(thread_handles);
    return 0;
}
```

使用互斥锁的全局求和函数

- 在配备双四核处理器的系统上，使用 $n=10^8$ 项的 π 计算程序的运行时间（单位：秒）。

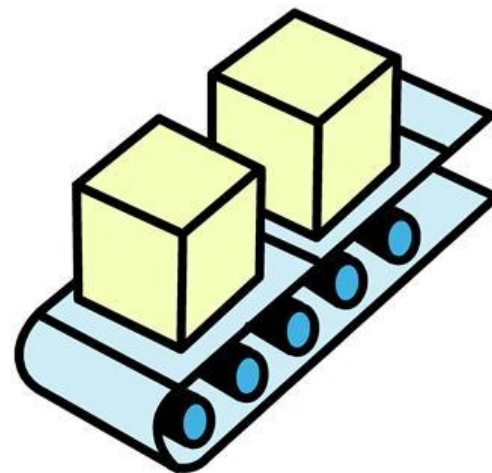
线程	忙等待	互斥锁
1	2.90	2.90
2	1.45	1.45
4	0.73	0.73
8	0.38	0.38
16	0.50	0.38
32	0.80	0.40
64	3.56	0.38

$$\frac{T_{\text{serial}}}{T_{\text{parallel}}} \approx \text{thread_count}$$

时间	flag	线程				
		0	1	2	3	4
0	0	临界区	忙等待	暂停	暂停	暂停
1	1	终止	临界区	暂停	忙等待	暂停
2	2	—	终止	暂停	忙等待	忙等待
⋮	⋮			⋮	⋮	⋮
?	2	—	—	临界区	暂停	忙等待

在采用忙等待、且线程数量多于核心数的情况下，可能出现的事件序列。

生产者-消费者同步与信号量



问题

- 忙等待会强制规定线程访问临界区的顺序。
- 而使用互斥锁时，访问顺序则是随机的，由系统决定。
- 在某些应用场景中，我们需要控制线程访问临界区的顺序。

互斥锁方案存在的问题

/* n和product_matrix被主线程共享并初始化。product_matrix被
* 初始化为单位矩阵。*/

```
void* Thread_work(void* rank) {  
    long my_rank = (long) rank;  
    matrix_t my_mat = Allocate_matrix(n);  
    Generate_matrix(my_mat);  
    pthread_mutex_lock(&mutex);  
    Multiply_matrix(product_mat, my_mat);  
    pthread_mutex_unlock(&mutex);  
    Free_matrix(&my_mat);  
    return NULL;  
} /* Thread_work */
```

使用Pthreads发送消息的初次尝试

```
/* 'messages'有char**类型。它是在main中分配的。 */
/* 每个条目在main中都被设置成NULL。 */
void *Send_msg(void* rank) {
    long my_rank = (long) rank;
    long dest = (my_rank + 1) % thread_count;
    long source = (my_rank + thread_count - 1) % thread_count;
    char* my_msg = malloc(MSG_MAX*sizeof(char));

    sprintf(my_msg, "Hello to %ld from %ld", dest, my_rank);
    messages[dest] = my_msg;

    if (messages[my_rank] != NULL)
        printf("Thread %ld > %s\n", my_rank, messages[my_rank]);
    else
        printf("Thread %ld > No message from %ld\n",
               my_rank, source);

    return NULL;
} /* Send_msg */
```


使用Pthreads发送消息的初次尝试

```
#define
_CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
const int MAX_THREADS = 1024;
const int MSG_MAX = 100;
int thread_count = 1;
char** messages;
void* Send_msg(void* rank) {
    long my_rank = (long)rank;
    long dest = (my_rank + 1) %
thread_count;
    long source = (my_rank +
thread_count - 1) % thread_count;
    char* my_msg =
(char*)malloc(MSG_MAX *
sizeof(char));
    sprintf(my_msg, "Hello to %ld from
%ld", dest, my_rank);
```

```
messages[dest] = my_msg;
if (messages[my_rank] != NULL)
    printf("Thread %ld > %s\n", my_rank,
messages[my_rank]);
else
    printf("Thread %ld > No message
from %ld\n", my_rank, source);
return NULL;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    thread_handles =
(pthread_t*)malloc(thread_count *
sizeof(pthread_t));
    messages =
(char**)malloc(thread_count *
sizeof(char*));
```

```
for (thread = 0; thread < thread_count;
thread++)
    messages[thread] = NULL;
for (thread = 0; thread < thread_count;
thread++)
    pthread_create(&thread_handles[thread],
(pthread_attr_t*)NULL, Send_msg,
(void*)thread);
for (thread = 0; thread < thread_count;
thread++) {
    pthread_join(thread_handles[thread],
NULL);
}
for (thread = 0; thread < thread_count;
thread++)
    free(messages[thread]);
free(messages);
free(thread_handles);
return 0;
}
```

各种信号量函数的语法

信号量并不属于Pthreads线程库的一部分；
你需要额外引入这个。

```
#include <semaphore.h>
```

```
int sem_init(  
    sem_t*      semaphore_p    /* out */,  
    int         shared          /* in */,  
    unsigned    initial_val     /* in */);
```

```
int sem_destroy(sem_t* semaphore_p /* in/out */);
```

```
int sem_post(sem_t* semaphore_p /* in/out */);
```

```
int sem_wait(sem_t* semaphore_p /* in/out */);
```

栅栏与条件变量



栅栏

- 同步线程以确保它们都到达程序中的同一位置，这种机制称为栅栏。
 - 。
- 在所有线程都到达栅栏之前，任何线程都无法越过该栅栏。

使用栅栏来测量最慢线程的耗时

```
/* 共享的 */  
double elapsed_time;  
.  
.  
.  
/* 私有的 */  
double my_start, my_finish, my_elapsed;  
.  
.  
.  
Synchronize threads;  
Store current time in my_start;  
/* 执行计时代码 */  
.  
.  
.  
Store current time in my_finish;  
my_elapsed = my_finish - my_start;  
  
elapsed = Maximum of my_elapsed values;
```

使用栅栏进行调试

指出我们想在程序中达到哪一个点；
栅栏；

```
if (my_rank == 0) {  
    printf("All threads reached this point\n");  
    fflush(stdout);  
}
```



忙等待与互斥锁

- 使用忙等待和互斥锁实现栅栏非常简单直接。
- 我们使用一个受互斥锁保护的共享计数器。
- 当计数器表明所有线程都已进入临界区时，线程就可以离开临界区。

忙等待与互斥锁

```
/* 通过主线程共享和初始化 */  
int counter; /* 初始化为0 */  
int thread_count;  
pthread_mutex_t barrier_mutex;  
.  
.  
.  
  
void* Thread_work(. . .) {  
    .  
    .  
    .  
    /* 栅栏 */  
    pthread_mutex_lock(&barrier_mutex);  
    counter++;  
    pthread_mutex_unlock(&barrier_mutex);  
    while (counter < thread_count);  
    .  
    .  
    .  
}
```

我们需要为每个栅栏实例设计一个counter变量, 否则很可能会出现问題。

使用Pthreads发送消息-互斥锁栅栏

```
#define
_CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
const int MAX_THREADS = 1024;
const int MSG_MAX = 100;
int thread_count = 1;
char** messages;
int counter = 0;
pthread_mutex_t barrier_mutex;
void* Send_msg(void* rank) {
    long my_rank = (long)rank;
    long dest = (my_rank + 1) % thread_count;
    long source = (my_rank + thread_count - 1)
        % thread_count;
    char* my_msg =
        (char*)malloc(MSG_MAX * sizeof(char));
    sprintf(my_msg, "Hello to %ld from %ld",
        dest, my_rank);
    messages[dest] = my_msg;
    pthread_mutex_lock(&barrier_mutex);
```

```
    counter++;
    pthread_mutex_unlock(&barrier_mutex);
    while (counter < thread_count);
    if (messages[my_rank] != NULL)
        printf("Thread %ld > %s\n", my_rank,
            messages[my_rank]);
    else
        printf("Thread %ld > No message from
            %ld\n", my_rank, source);
    return NULL;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    thread_handles =
        (pthread_t*)malloc(thread_count *
            sizeof(pthread_t));
    messages = (char**)malloc(thread_count *
        sizeof(char*));
    for (thread = 0; thread < thread_count;
```

```
        thread++)
        messages[thread] = NULL;
    pthread_mutex_init(&barrier_mutex,
        NULL);
    for (thread = 0; thread < thread_count;
        thread++)
        pthread_create(&thread_handles[thread],
            (pthread_attr_t*)NULL, Send_msg,
            (void*)thread);
    for (thread = 0; thread < thread_count;
        thread++) {
        pthread_join(thread_handles[thread],
            NULL);
    }
    for (thread = 0; thread < thread_count;
        thread++)
        free(messages[thread]);
    free(messages);
    pthread_mutex_destroy(&barrier_mutex);
    free(thread_handles);
    return 0;
}
```

使用信号量实现栅栏

```
/* 共享变量 */
int counter; /* 初始化为0 */
sem_t count_sem; /* 初始化为1 */
sem_t barrier_sem; /* 初始化为0 */
. . .
void* Thread_work (...) {
    . . .
    /* 栅栏 */
    sem_wait(&count_sem);
    if (counter == thread_count - 1) {
        counter = 0;
        sem_post(&count_sem);
        for (j = 0; j < thread_count - 1; j++)
            sem_post(&barrier_sem);
    } else {
        counter++;
        sem_post(&count_sem);
        sem_wait(&barrier_sem);
    }
    . . .
}
```

使用Pthreads发送消息-信号量栅栏

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#include <pthread.h>
const int MAX_THREADS = 1024;
const int MSG_MAX = 100;
int thread_count = 1;
char** messages;
int counter = 0;
sem_t count_sem, barrier_sem;
void* Send_msg(void* rank) {
    long my_rank = (long)rank;
    long dest = (my_rank + 1) % thread_count;
    long source = (my_rank + thread_count - 1) %
thread_count;
    char* my_msg = (char*)malloc(MSG_MAX *
sizeof(char));
    sprintf(my_msg, "Hello to %ld from %ld",
dest, my_rank);
    messages[dest] = my_msg;
    sem_wait(&count_sem);
    if (counter == thread_count - 1) {
        counter = 0;
        sem_post(&count_sem);
```

```
        for (int i = 0; i < thread_count - 1; i++)
            sem_post(&barrier_sem);
    }
    else {
        counter++;
        sem_post(&count_sem);
        sem_wait(&barrier_sem);
    }
    if (messages[my_rank] != NULL)
        printf("Thread %ld > %s\n", my_rank,
messages[my_rank]);
    else
        printf("Thread %ld > No message from %ld\n",
my_rank, source);
    return NULL;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    thread_handles =
(pthread_t*)malloc(thread_count *
sizeof(pthread_t));
    messages = (char**)malloc(thread_count *
```

```
sizeof(char*));
    for (thread = 0; thread < thread_count;
thread++)
        messages[thread] = NULL;
    sem_init(&count_sem, 0, 1);
    sem_init(&barrier_sem, 0, 0);
    for (thread = 0; thread < thread_count;
thread++)
        pthread_create(&thread_handles[thread],
(pthread_attr_t*)NULL, Send_msg,
(void*)thread);
    for (thread = 0; thread < thread_count;
thread++) {
        pthread_join(thread_handles[thread], NULL);
    }
    for (thread = 0; thread < thread_count;
thread++)
        free(messages[thread]);
    free(messages);
    sem_destroy(&count_sem);
    sem_destroy(&barrier_sem);
    free(thread_handles);
    return 0;
}
```

条件变量

- 条件变量是一种数据对象，它允许线程暂停执行，直到某个特定事件或条件发生。
- 当该事件或条件发生时，另一个线程可以发送信号，让该线程“唤醒”。
- 条件变量总是与互斥锁关联使用。

条件变量

```
    锁住互斥锁;  
    if 条件发生  
        signal thread(s);  
    else {  
        解锁互斥锁和块;  
        /* 当线程被解锁时, 互斥锁也被重新上锁 */  
    }  
    unlock mutex;
```

使用条件变量实现栅栏

```
/* 共享的 */
int counter = 0;
pthread_mutex_t mutex;
pthread_cond_t cond_var;
. . .
void* Thread_work(. . .) {
    . . .
    /* 栅栏 */
    pthread_mutex_lock(&mutex);
    counter++;
    if (counter == thread_count) {
        counter = 0;
        pthread_cond_broadcast(&cond_var);
    } else {
        while (pthread_cond_wait(&cond_var, &mutex) != 0);
    }
    pthread_mutex_unlock(&mutex);
    . . .
}
```

使用Pthreads发送消息-条件变量栅栏

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
const int MAX_THREADS = 1024;
const int MSG_MAX = 100;
int thread_count = 1;
char** messages;
int counter = 0;
pthread_mutex_t mutex;
pthread_cond_t cond_var;
void* Send_msg(void* rank) {
    long my_rank = (long)rank;
    long dest = (my_rank + 1) % thread_count;
    long source = (my_rank + thread_count - 1) %
        thread_count;
    char* my_msg = (char*)malloc(MSG_MAX *
        sizeof(char));
    sprintf(my_msg, "Hello to %ld from %ld",
        dest, my_rank);
    messages[dest] = my_msg;
    pthread_mutex_lock(&mutex);
    counter++;
    if (counter == thread_count) {
```

```
        counter = 0;
        pthread_cond_broadcast(&cond_var);
    }
    else
        pthread_cond_wait(&cond_var, &mutex);
    pthread_mutex_unlock(&mutex);
    if (messages[my_rank] != NULL)
        printf("Thread %ld > %s\n", my_rank,
            messages[my_rank]);
    else
        printf("Thread %ld > No message from %ld\n",
            my_rank, source);
    return NULL;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    thread_handles =
        (pthread_t*)malloc(thread_count *
            sizeof(pthread_t));
    messages = (char**)malloc(thread_count *
        sizeof(char*));
```

```
    for (thread = 0; thread < thread_count;
        thread++)
        messages[thread] = NULL;
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&cond_var, NULL);
    for (thread = 0; thread < thread_count;
        thread++)
        pthread_create(&thread_handles[thread],
            (pthread_attr_t*)NULL, Send_msg,
            (void*)thread);
    for (thread = 0; thread < thread_count;
        thread++) {
        pthread_join(thread_handles[thread], NULL);
    }
    for (thread = 0; thread < thread_count;
        thread++)
        free(messages[thread]);
    free(messages);
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&cond_var);
    free(thread_handles);
    return 0;
}
```

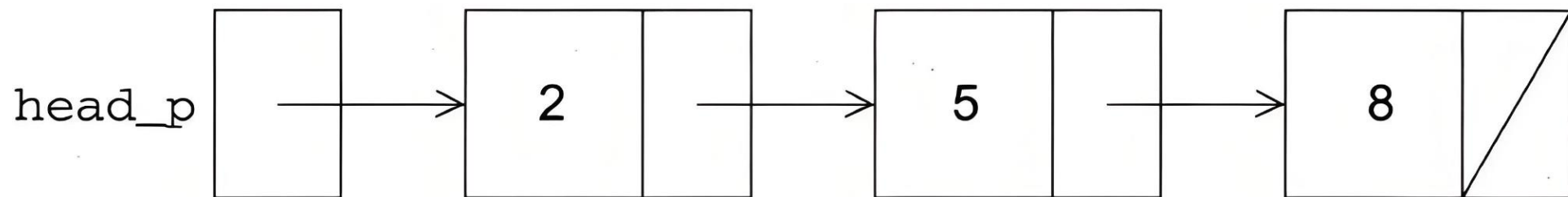
读写锁



控制对大型共享数据结构的访问

- 我们来看一个例子。
- 假设该共享数据结构是一个有序整型链表，需要实现的核心操作包括：查找成员、插入和删除。

链表

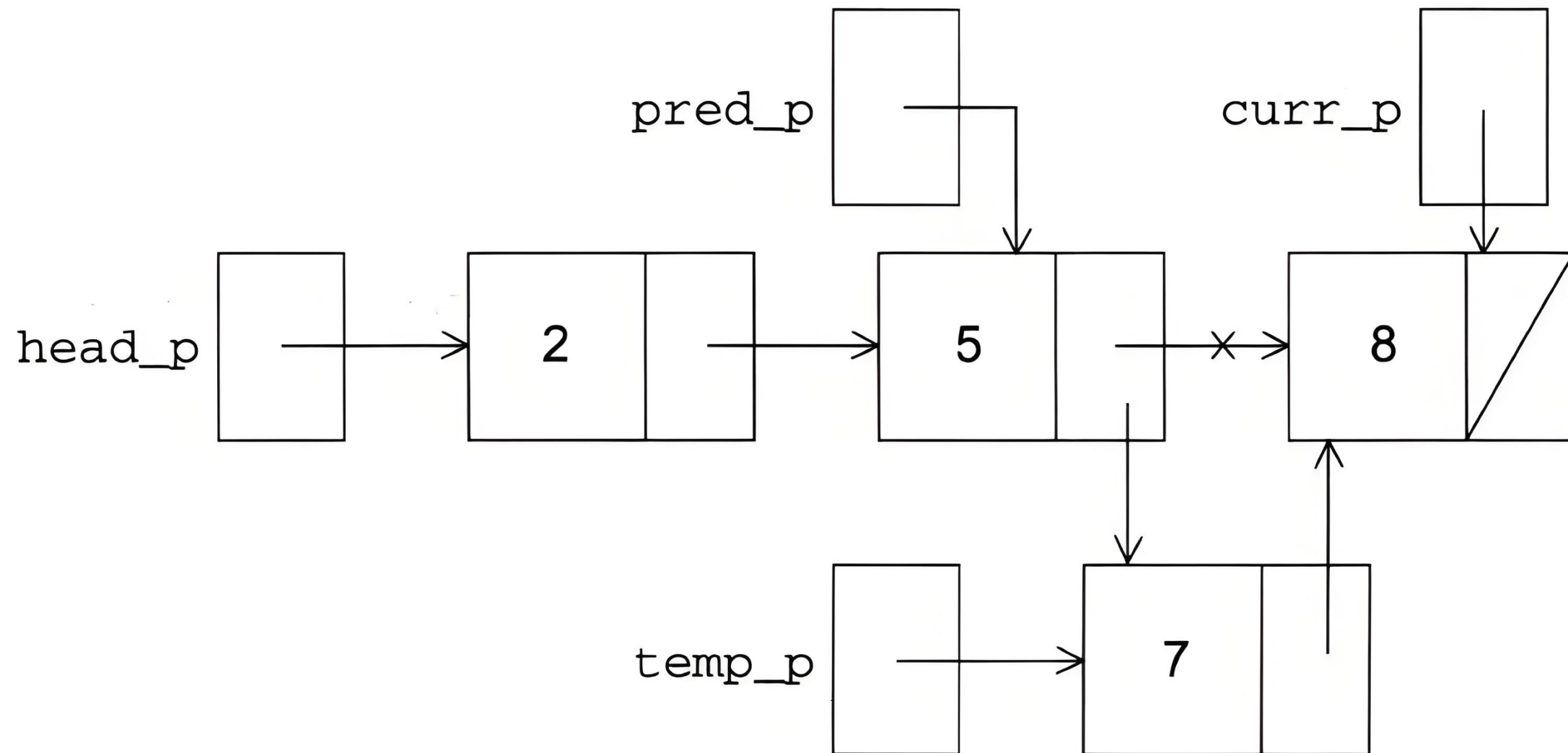


```
struct list_node_s {  
    int data;  
    struct list_node_s* next;  
}
```

链表成员判定

```
int Member(int value, struct list_node_s* head_p) {  
    struct list_node_s* curr_p = head_p;  
  
    while (curr_p != NULL && curr_p->data < value)  
        curr_p = curr_p->next;  
  
    if (curr_p == NULL || curr_p->data > value) {  
        return 0;  
    } else {  
        return 1;  
    }  
} /* Member */
```

向链表中插入新节点



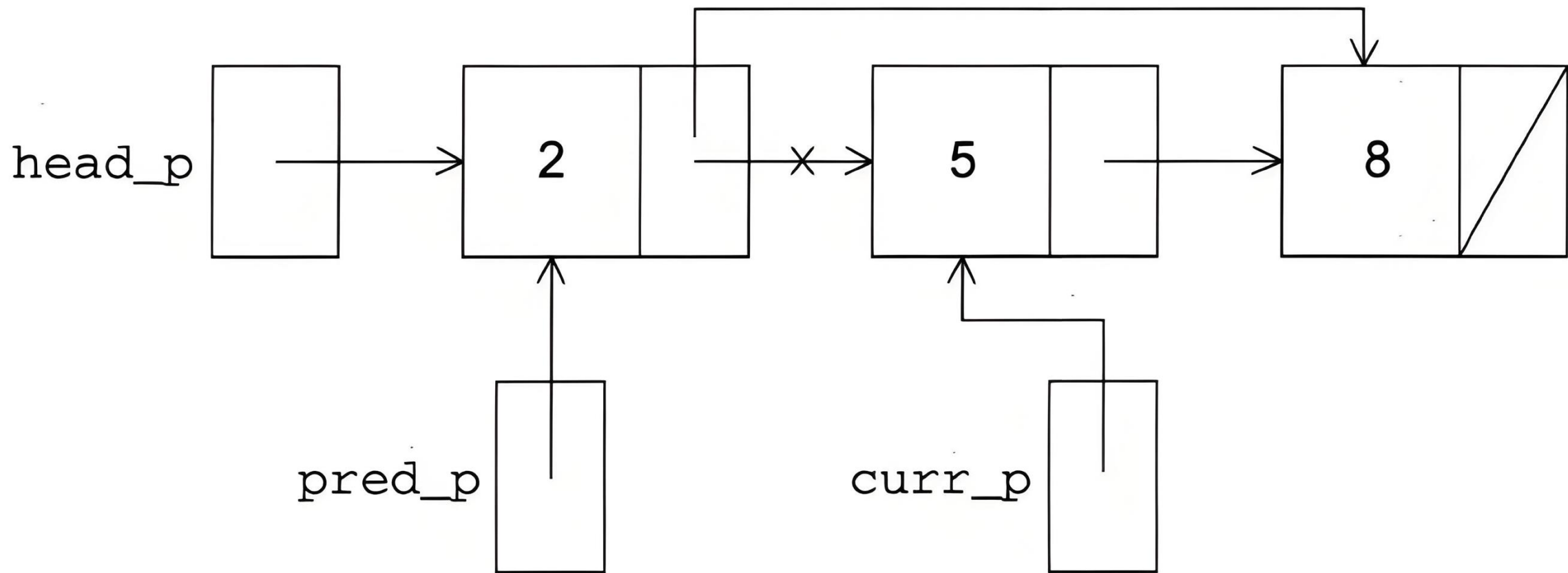
向链表中插入新节点

```
int Insert(int value, struct list_node_s** head_pp) {
    struct list_node_s* curr_p = *head_pp;
    struct list_node_s* pred_p = NULL;
    struct list_node_s* temp_p;

    while (curr_p != NULL && curr_p->data < value) {
        pred_p = curr_p;
        curr_p = curr_p->next;
    }

    if (curr_p == NULL || curr_p->data > value) {
        temp_p = malloc(sizeof(struct list_node_s));
        temp_p->data = value;
        temp_p->next = curr_p;
        if (pred_p == NULL) /* 第一个新的节点 */
            *head_pp = temp_p;
        else
            pred_p->next = temp_p;
        return 1;
    } else { /* 已经在链表中的值 */
        return 0;
    }
} /* Insert */
```

从链表中删除节点



从链表中删除节点

```
int Delete(int value, struct list_node_s** head_pp) {
    struct list_node_s* curr_p = *head_pp;
    struct list_node_s* pred_p = NULL;

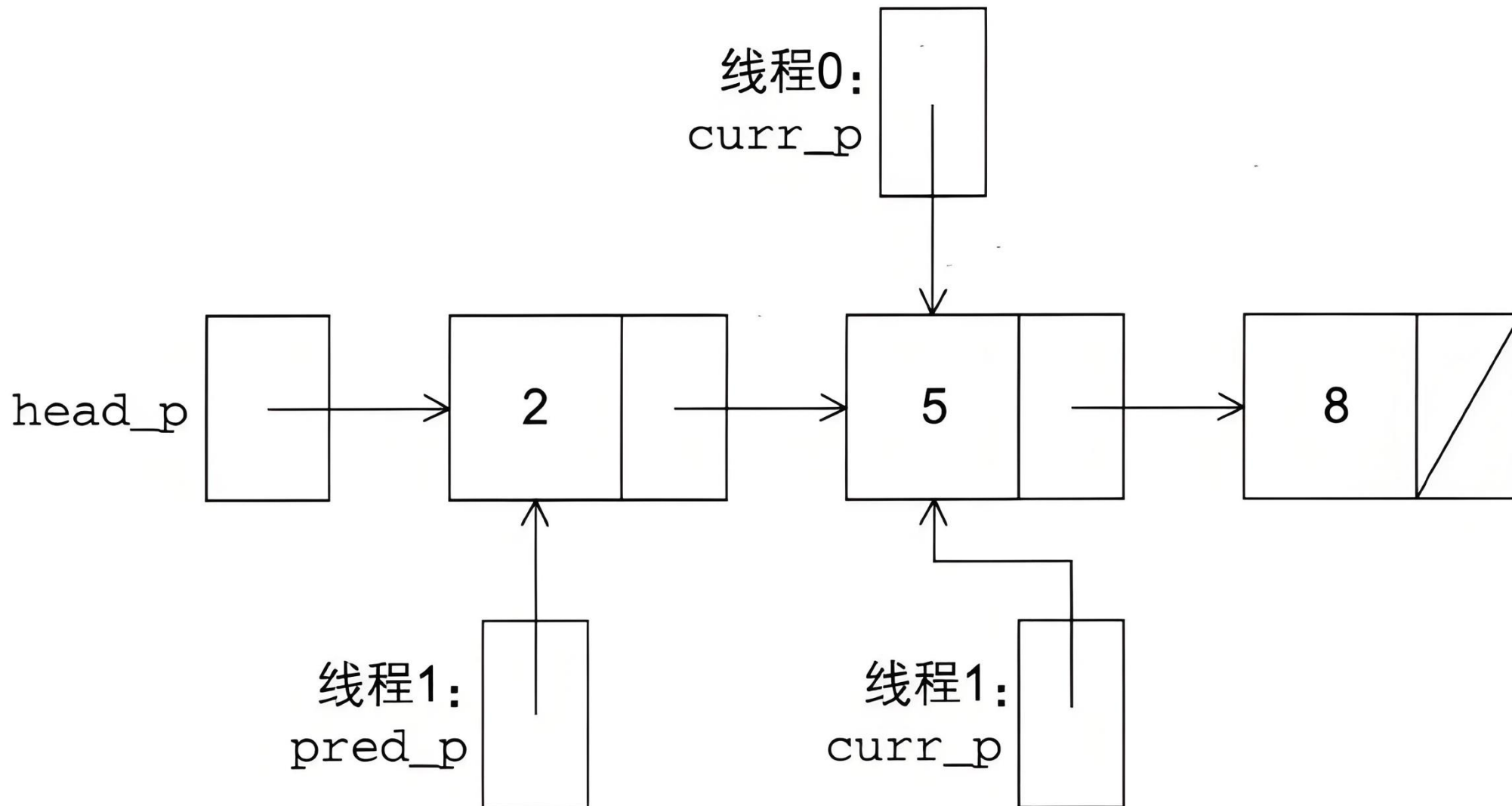
    while (curr_p != NULL && curr_p->data < value) {
        pred_p = curr_p;
        curr_p = curr_p->next;
    }

    if (curr_p != NULL && curr_p->data == value) {
        if (pred_p == NULL) { /* 删除链表中的第一个节点 */
            *head_pp = curr_p->next;
            free(curr_p);
        } else {
            pred_p->next = curr_p->next;
            free(curr_p);
        }
        return 1;
    } else { /* 不在链表内的值 */
        return 0;
    }
} /* Delete */
```

多线程链表

- 我们来尝试在一个Pthreads程序中使用这些函数。
- 为了共享对该链表的访问，我们可以将`head_p`定义为全局变量。
- 这样可以简化`Member`、`Insert`和`Delete`函数的函数头，因为我们无需再传入`head_p`或指向`head_p`的指针，只需要传入要操作的目标值即可。

两个线程同时访问



解决方案#1

- 一个显而易见的解决方案是：每当线程试图访问链表时，直接对其加锁。
- 对这三个函数的每次调用都可以通过互斥锁（mutex）进行保护。

```
Pthread_mutex_lock(&list_mutex);
```

```
Member(value);
```

```
Pthread_mutex_unlock(&list_mutex);
```

代替调用Member(value)

存在的问题

- 我们正在对链表的访问进行串行化处理。
- 如果我们绝大多数操作都是成员查询（Member），那么就会错失利用并行性的机会。
- 另一方面，如果大部分操作都是插入（Insert）和删除（Delete），那么这种方案可能就是最佳选择，因为此时大多数操作都需要对链表进行串行访问，而且该方案实现起来无疑非常简单。

解决方案#2

- 与其锁定整个链表，我们可以尝试单独锁定每个节点。
- 这是一种更细粒度的实现方案。

```
struct list_node_s {  
    int data;  
    struct list_node_s* next;  
    pthread_mutex_t mutex;  
}
```

存在的问题

- 这比原来的成员查询函数要复杂得多。
- 同时它的执行速度也会慢很多，因为通常情况下，每次访问一个节点时，都必须对互斥锁进行加锁和解锁操作。
- 而为每个节点增加一个互斥锁字段，将会显著增加链表所需的存储空间。

采用每个链表节点配备一个互斥锁的方式实现成员查询操作

```
int Member(int value) {
    struct list_node_s *temp_p, *old_temp_p;
    Pthread_mutex_lock(&head_p_mutex);
    /* 如果表非空，则获取与第一个节点关联的互斥锁 */
    if (temp_p != NULL)
        Pthread_mutex_lock(temp_P->mutex);
    /* 不再需要head_p互斥锁 */
    Pthread_mutex_unlock(&head_P_mutex);
    while (temp_P != NULL && temp_P->data < value) {
        if (temp_P->next != NULL)
            Pthread_mutex_lock(&(temp_P->next->mutex));
        /* 进入下一个元素 */
        old_temp_P = temp_P;
        temp_P = temp_P->next;
        /* 现在解锁前一个元素的互斥锁 */
        Pthread_mutex_unlock(&(old_temp_P->mutex));
    }
    if (temp_P == MULL || temp_P->data > value) {
        if (temp_P != MULL)
            Pthread_mutex_unlock(&temp_P->mutex);
        return 0;
    } else { /* temp_P != NULL && temp_P->data == value */
        Pthread_mutex_unlock(&temp_P->mutex);
        return 1;
    }
} /* Member */
```

Pthreads读写锁

- 我们的这两种多线程链表实现，都没有充分利用执行成员查询操作的线程同时访问任意节点的潜在并行性。
- 第一种方案在任意时刻只允许一个线程访问整个链表。
- 第二种方案在任意时刻只允许一个线程访问任意指定节点。

Pthreads读写锁

- 读写锁有点类似于互斥锁，区别在于它提供两种加锁函数。
- 第一种加锁函数以读模式对读写锁加锁，而第二种则以写模式对其加锁。

Pthreads读写锁

- 因此，多个线程可以通过调用读锁函数同时获取该锁，而只有一个线程能通过调用写锁函数获取该锁。
- 如此一来，如果已有线程以读模式持有锁，那么任何想要以写模式获取锁的线程，都会在调用写锁函数时阻塞。
- 如果有任意一个线程以写模式持有该锁，那么所有想要以读模式或写模式获取该锁的线程，都将在各自的加锁函数调用中进入阻塞状态。



保护我们的链表函数

```
pthread_rwlock_rdlock(&rwlock);
Member(value);
pthread_rwlock_unlock(&rwlock);
. . .
pthread_rwlock_wrlock(&rwlock);
Insert(value);
pthread_rwlock_unlock(&rwlock);
. . .
pthread_rwlock_wrlock(&rwlock);
Delete(value);
pthread_rwlock_unlock(&rwlock);
```

链表性能

链表次数：100000次 / 线程，99.9%Member，0.05%Insert，0.05%Delete

实现	线程的数量			
	1	2	4	8
读写锁	0.213	0.123	0.098	0.115
整个链表一个互斥锁	0.211	0.450	0.385	0.457
每个节点一个互斥锁	1.680	5.700	3.450	2.700

链表次数：100000次 / 线程，80%Member，10%Insert，10%Delete

实现	线程的数量			
	1	2	4	8
读写锁	2.48	4.97	4.69	4.71
整个链表一个互斥锁	2.50	5.13	5.04	5.11
每个节点一个互斥锁	12.00	29.60	17.00	12.00

缓存、缓存一致性与伪共享

- 回想一下，芯片设计师在处理器中增加了速度相对较快的存储器块，称为高速缓存（cache memory）。
- 高速缓存的使用会对共享内存产生巨大影响。
- 当处理器核心试图更新一个不在高速缓存中的变量时，就会发生写缺失，此时它必须访问主存。

Pthreads矩阵-向量乘法

```
void *Pth_mat_vect(void* rank) {
    long my_rank = (long) rank;
    int i, j;
    int local_m = m/thread_count;
    int my_first_row = my_rank*local_m;
    int my_last_row = (my_rank+1)*local_m - 1;

    for (i = my_first_row; i <= my_last_row; i++) {
        y[i] = 0.0;
        for (j = 0; j < n; j++)
            y[i] += A[i][j]*x[j];
    }

    return NULL;
} /* Pth_mat_vect */
```

Pthreads矩阵-向量乘法

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
int thread_count;
const int m = 2520 * 4;
const int n = m;
double A[m][n];
double x[n];
double y[m];
void* Pth_mat_vect(void* rank) {
    long my_rank = (long)rank;
    int local_m = m / thread_count;
    int my_first_row = my_rank * local_m;
    int my_last_row = (my_rank + 1) * local_m - 1;
    for (int i = my_first_row; i <= my_last_row; i++) {
        y[i] = 0.0;
        for (int j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    }
    return NULL;
}
```

```
}
int main(int argc, char* argv[]) {
    long thread;
    double start, finish, elapsed;
    thread_count = atoi(argv[1]);
    pthread_t* thread_handles =
        (pthread_t*)malloc(thread_count * sizeof(pthread_t));
    start = clock();
    for (thread = 0; thread < thread_count; thread++)
        pthread_create(&thread_handles[thread], NULL,
            Pth_mat_vect, (void*)thread);
    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);
    finish = clock();
    elapsed = (finish - start) / CLOCKS_PER_SEC;
    printf("Elapsed time = %f seconds\n", elapsed);
    free(thread_handles);
    return 0;
}
```

矩阵-向量乘法的运行时间和效率

线程	矩阵维度					
	8 000 000 × 8		8 000 × 8 000		8 × 8 000 000	
	时间	效率	时间	效率	时间	效率
1	0.393	1.000	0.345	1.000	0.441	1.000
2	0.217	0.906	0.188	0.918	0.300	0.735
4	0.139	0.707	0.115	0.750	0.388	0.290

(时间单位为秒)

缓存伪共享

■ 缓存行 (Cache Line)

- ▶ CPU缓存与内存交换的最小单位，现代x86/ARM通常为64字节。
- ▶ 访问一个变量时，会把它所在的整个64字节块加载到缓存。

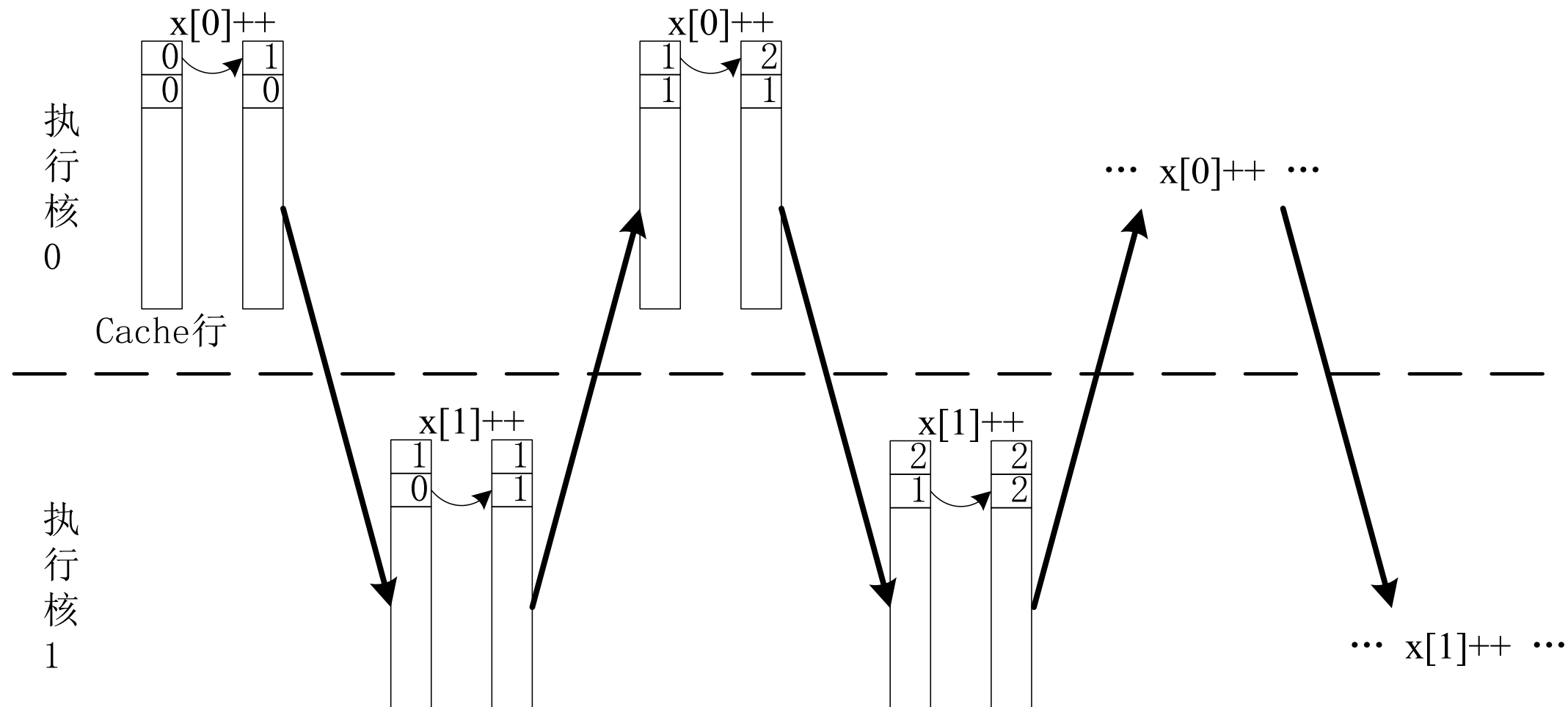
■ 缓存一致性协议

- ▶ 多核间通过缓存一致性协议保证缓存一致。
- ▶ 当一个核心修改缓存行中的任意一个变量，会将该行标记为Modified（脏），并向所有其他核心发送无效化（Invalidate）消息，强制它们的副本失效。

■ 伪共享的发生

- ▶ 线程A修改变量X，线程B修改变量Y。
- ▶ X、Y逻辑无关，但在内存中相邻，落入同一个64字节缓存行。
- ▶ A修改X→整个缓存行失效→B访问Y时必须重新从内存加载→性能雪崩。

伪共享引起的Cache行乒乓现象



基于数组的Pthreads全局求和

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <pthread.h>
long thread_count = 1;
long long n = 1000000000;
double sum = 0.0;
double my_sum[1000];
void* Thread_sum(void* rank) {
    long my_rank = (long)rank;
    long long my_n = n / thread_count;
    long long my_first_i = my_n *
my_rank;
    long long my_last_i = my_first_i +
my_n;
    double factor = my_first_i % 2 == 0 ?
1.0 : -1.0;
    my_sum[my_rank] = 0.0;
    for (long long i = my_first_i; i <
my_last_i; i++, factor = -factor)
```

```
my_sum[my_rank] += factor / (2 * i +
1);
return NULL;
}
int main(int argc, char* argv[]) {
    long thread;
    pthread_t* thread_handles;
    double start, finish, elapsed;
    if (argc > 1)
        thread_count = atoi(argv[1]);
    thread_handles =
(pthread_t*)malloc(thread_count *
sizeof(pthread_t));
    start = clock();
    for (thread = 0; thread < thread_count;
thread++)
        pthread_create(&thread_handles[thread],
NULL, Thread_sum,
(void*)thread);
    for (thread = 0; thread < thread_count;
```

```
thread++)
    pthread_join(thread_handles[thread],
NULL);
    finish = clock();
    elapsed = (finish - start) /
CLOCKS_PER_SEC;
    for (int i = 0; i < thread_count; i++)
        sum += my_sum[i];
    sum = 4.0 * sum;
    printf("Multi-threaded estimate of pi
= %.15f\n", sum);
    printf("Elapsed time = %f seconds\n",
elapsed);
    printf("Math library estimate of pi =
%.15f\n", 4.0 * atan(1.0));
    free(thread_handles);
    return 0;
}
```

线程安全



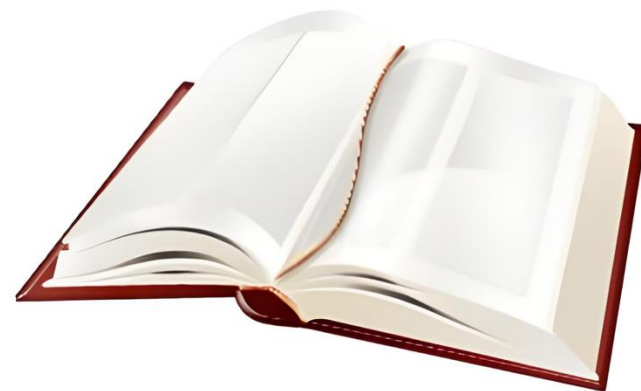
线程安全

- 如果一段代码能够被多个线程同时执行而不会引发问题，那么这段代码就是线程安全的。



示例

- 假设我们希望使用多个线程对一份由普通英文文本构成的文件进行分词处理。
- 这里的词元指的是由空白符（空格、制表符或换行符）与文本其他部分分隔开的连续字符序列。



简单方案

- 将输入文件按行划分，并以轮询调度的方式把这些行分配给各个线程。
- 第1行分配给线程0，第2行分配给线程1，.....，第t行分配给线程t，第t+1行再分配给线程0，依此类推。
- 我们可以使用信号量来实现对输入行的串行化访问。
- 在线程读取完一行输入后，便可通过`strtok`函数对该行进行分词处理。

strtok函数

- 第一次调用该函数时，字符串参数应传入需要进行分词处理的文本。
 - 我们的输入行。
- 在后续的调用中，第一个参数应设为NULL。
- 在第一次调用时，`strtok`会缓存一个指向该字符串的指针；而在后续调用中，它会从这份缓存的副本中依次返回后续的各个词元。

```
char* strtok(  
    char*          string      /* in/out */,  
    const char*  separators /* in      */);
```

多线程分词器

```
void *Tokenize(void* rank) {
    long my_rank = (long) rank;
    int count;
    int next = (my_rank + 1) % thread_count;
    char *fg_rv;
    char my_line[MAX];
    char *my_string;
    sem_wait(&sems[my_rank]);
    fg_rv = fgets(my_line, MAX, stdin);
    sem_post(&sems[next]);
    while (fg_rv != NULL) {
        printf("Thread %ld > my line = %s", my_rank, my_line);
        count = 0;
        my_string = strtok(my_line, " \t\n");
        while (my_string != NULL) {
            count++;
            printf("Thread %ld > string %d = %s\n",
                my_rank, count, my_string);
            my_string = strtok(NULL, " \t\n");
        }
        sem_wait(&sems[my_rank]);
        fg_rv = fgets(my_line, MAX, stdin);
        sem_post(&sems[next]);
    }
    return NULL;
} /* Tokenize */
```


单线程运行

- 它能够对输入流进行正确分词。

Pease porridge hot.

Pease porridge cold.

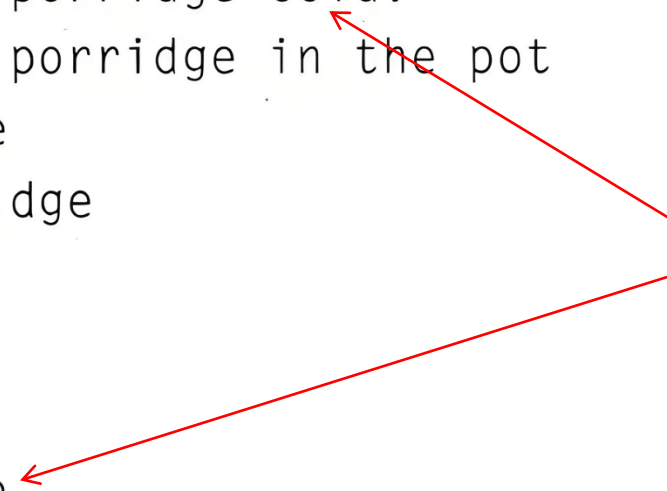
Pease porridge in the pot

Nine days old.

双线程运行

```
Thread 0 > my line = Pease porridge hot.  
Thread 0 > string 1 = Pease  
Thread 0 > string 2 = porridge  
Thread 0 > string 3 = hot.  
Thread 1 > my line = Pease porridge cold.  
Thread 0 > my line = Pease porridge in the pot  
Thread 0 > string 1 = Pease  
Thread 0 > string 2 = porridge  
Thread 0 > string 3 = in  
Thread 0 > string 4 = the  
Thread 0 > string 5 = pot  
Thread 1 > string 1 = Pease  
Thread 1 > my line = Nine days old.  
Thread 1 > string 1 = Nine  
Thread 1 > string 2 = days  
Thread 1 > string 3 = old.
```

哎呀!



发生了什么？

- `strtok`通过声明一个具有静态存储期的变量来缓存输入行。
- 这使得存储在该变量中的值在多次函数调用之间保持不变。
- 但对我们而言很麻烦的是：这个被缓存的字符串是全局共享的，而不是线程私有的。

发生了什么？

- 因此，线程0在处理输入第3行时调用的 `strtok`，显然覆盖了线程1在处理第2行时传入的内容。
- 所以`strtok`函数不是线程安全的。



其他非线程安全的C库函数

- 遗憾的是，C标准库中的函数并非线程安全，这种情况并不少见。
- `stdlib.h`中的随机数生成函数`random`。
- `time.h`中的时间转换函数`localtime`。

“可重入”（线程安全）函数

- 在某些情况下，C语言标准为函数提供了可替代的线程安全版本。

```
char* strtok_r(  
    char*          string      /* in/out */,  
    const char*   separators /* in */,  
    char**         saveptr_p  /* in/out */);
```

小结

- 共享内存编程中的线程，类似于分布式内存编程中的进程。
- 然而，线程通常比完整的进程更轻量级。
- 在Pthreads程序中，所有线程都可以访问全局变量，而局部变量通常对于运行该函数的线程而言是私有的。

小结

- 当多个线程试图访问共享资源（如共享变量或共享文件）时会产生不确定性，若其中至少有一次访问是更新操作，且这些访问可能导致错误，这种情况就称为**竞态条件**。

小结

- **临界区**是一段更新共享资源的代码块，该资源同一时刻只能被一个线程执行更新操作。
- 因此，临界区内代码的执行，实际上应当以串行方式运行。

小结

- 可以采用忙等待的方式，借助一个标志变量和一个空循环体的while循环，来避免对临界区的访问冲突。
- 这种方式会极大地浪费CPU时钟周期。
- 并且如果开启了编译器优化，该方法还可能不可靠。

小结

- **互斥锁** (mutex) 也可以用来避免对临界区的访问冲突。
- 可以把它看作是临界区上的一把锁，因为互斥锁的作用就是实现对临界区的互斥访问。

小结

- **信号量**是避免临界区访问冲突的第三种方法。
- 它由一个无符号整型变量，以及sem_wait和sem_post两个操作共同构成。
- 信号量的功能比互斥锁更强大，因为它可以被初始化为任意非负整数。

小结

- **栅栏** (barrier) 是程序中的一个节点，线程会在此处阻塞等待，直到所有线程都到达该位置。
- **读写锁** (read-write lock) 适用于以下场景：多个线程同时读取某个数据结构是安全的，但如果某个线程需要修改或写入该数据结构，则在修改期间只允许该线程访问该数据结构。

小结

- 一些C语言函数通过将变量声明为静态（static），在多次调用之间缓存数据，当多个线程调用该函数时就会引发错误。
- 这类函数不是线程安全的。